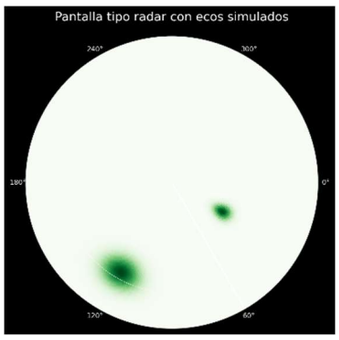
ICOM2025 – 1er Parcial

9 de octubre de 2025

Notas:

1. Uso de prácticos: **se pueden utilizar los trabajos prácticos propios realizados.**
2. **NO UTILIZAR** sistemas de ayuda tipo chatGPT

Problema 1: **Imagen Radial**



En la visualización de datos de video en tiempo real provenientes de un radar (meteorológico, de tráfico aéreo o de defensa), se emplea el protocolo **ASTERIX, categoría 240**.

Este protocolo representa la información recibida como un conjunto de registros en coordenadas polares, con origen en la posición del radar.

Cada registro describe un **sector angular de la imagen** mediante la siguiente estructura:

```
using Pixel = uint8_t;

struct AX240Record {
    float rangeStart;       // Rango inicial del sector
    float rangeStep;        // Paso de discretización del rango
                            // (ligado a la duración de la celda de muestreo)
    float aziStart;         // Azimuth inicial en grados
    float aziEnd;           // Azimuth final en grados
    std::vector<Pixel> pixeles; // Pixeles del sector, en orden creciente
                            // de distancia al radar
};
```

Dado que no se cuenta con datos reales de radar, se desea **simular registros** a partir de una imagen estática representada por una matriz cuadrada de píxeles de **SIZE** x **SIZE**:

```
using Image = std::vector<std::vector<Pixel>>;
```

Objetivo: Se pide implementar las funciones:

```
// lee y retorna la imagen que se encuentra almacenada en el archivo fileName.
// dentro del archivo, la imagen se encuentra organizada como
// ancho, alto
// pixel0, pixel1, pixel2, ...
Image readImage(const string &fileName);

// Función que debe retornar un std::vector de SIZE/2 píxeles,(donde SIZE es el ancho/alto
// de la imagen) que corresponden a la secuencia de valores obtenidos radialmente desde el
// centro de la imagen en la dirección especificada por el ángulo azimuth.
std::vector<Pixel> getRadial(const Image &img, float azimuth);
```

En el archivo **`Apellido_Nombre_Ejer1.cpp`** se encuentra un esqueleto del programa

Problema 2: **La princesa y el caballero**

Un caballero en brillante armadura fue secuestrado por un rey enemigo y lo mantienen cautivo en la celda más interna de una mazmorra. La princesa quiere rescatar al caballero (princesa empoderada, ¿vió?) y como el camino está lleno de peligros, solicita la ayuda de su hechicero/a (o sea usted) para encontrar la mejor forma de cumplir su cometido. La mazmorra se representa como una matriz de **`N x M`** celdas y el caballero se encuentra encerrado en la celda **`{N-1,M-1}`**. La princesa comienza su camino por la celda **`{0,0}`** con una cierta cantidad de energía, y en cada celda la princesa puede

ganar o perder energía (representada por números enteros positivos o negativos de la matriz) y en cada celda, solo puede moverse hacia la **derecha** o hacia **abajo**. En su camino, **nunca** debe ocurrir que la energía de la princesa sea **menor o igual a 0**, porque entonces estaría muerta. Dado que la princesa tiene … digamos ciertas intenciones para con el caballero cuando lo rescate, se le solicita que encuentre el camino óptimo por el cual se llega al caballero con la **mayor cantidad de energía restante**.

En el archivo **Apellido_Nombre_Ejer2.cpp** se le entrega un main y el prototipo de la función que deberá implementar:
int camino_optimo(int maz[N][M], celda c, int energia);

Esta función recursiva, recibe la matriz **maz**, la celda **c** a partir del cual se debe continuar el recorrido y la **energia** actual de la princesa. Debe retornar la energía final del recorrido óptimo.

Nota: debido a la regla de que la energía en cada momento no puede ser <= 0, el camino óptimo depende de la energía inicial.

Ejemplo de camino óptimo, comenzando con energía 7:

-2 → -3 → 3
↓
-5 -10 1
↓
10 30 -5

Problema 3: **xSort**

En este ejercicio, se les proporcionará el esqueleto de un programa escrito en C++ (**`Apellido_Nombre_Ejer3.cpp`**) que implementa un algoritmo de ordenamiento específico para enteros sin signo y lo compara con **`std::sort`**. Su tarea será definir las funciones **`getDigit`** y **`xSort`**, respetando sus declaraciones. El resto del código, incluyendo el **`main`** y las constantes globales, no deberá modificarse.

Detalles sobre las funciones:

`unsigned int getDigit(unsigned int num, unsigned int shift)`

Esta función debe extraer y devolver el dígito de **`DIGIT_BITS`** bits que comienza en la posición **shift** de un número **`num`**. Por ejemplo, si **`DIGIT_BITS = 8`** (base 256), el valor devuelto estará comprendido entre 0 y 255.

`void xSort(vector<unsigned int> & arr)`

Esta función debe ordenar el vector **`arr`** utilizando el algoritmo explicado más adelante, procesando los números dígito por dígito, de menos significativo a más significativo. La separación en dígitos estará dada por la base utilizada, definida indirectamente mediante la cantidad de bits que ocupa cada dígito (**`DIGIT_BITS`**).

Explicación de xSort con un ejemplo (base 10)

Para entender el algoritmo, consideremos un ejemplo sencillo con números de 2 dígitos en base 10. Supongamos que tenemos el siguiente arreglo de números: {64, 23, 91, 12, 45}

xSort ordenará los números procesando un dígito a la vez, comenzando por el dígito menos significativo.

Usaremos **buckets** (baldes/contenedores) para agrupar los números según el valor del dígito que estemos procesando.

Ejemplo, paso 1: Ordenar por el dígito menos significativo (unidades)

Primero, contamos las ocurrencias de cada dígito (valores posibles entre 0 y 9) en un arreglo de 10 buckets (porque cada dígito puede tener sólo 10 posibles valores):

- Bucket[0] = 0 (ningún número termina en 0)
- Bucket[1] = 1 (un número, el 91, termina en 1)
- Bucket[2] = 1 (un número, el 12, termina en 2)

- Bucket[3] = 1 (un número, el 23, termina en 3)
- Bucket[4] = 1 (un número, el 64, termina en 4)
- Bucket[5] = 1 (un número, el 45, termina en 5)
- Bucket[6 a 9] = 0

Luego, calculamos las posiciones acumuladas sumando los valores de los buckets:

- Bucket[0] = Bucket[0] (es decir, no cambia)
- Bucket[1] = Bucket[1] + Bucket[0] = 1
- Bucket[2] = Bucket[2] + Bucket[1] = 2
- Bucket[3] = Bucket[3] + Bucket[2] = 3

- Bucket[4] = Bucket[4] + Bucket[3] = 4
- Bucket[5] = Bucket[5] + Bucket[4] = 5
- Bucket[6 a 9] = 5 (ya que todos valían 0)

Ahora, colocamos los números en un arreglo auxiliar según el dígito de las unidades, recorriendo el arreglo original de derecha a izquierda para mantener el orden relativo (recuerde que el arreglo original era {64, 23, 91, 12, 45}):

- Para 45 (dígito 5): Posición = Bucket[5] - 1 = 4. Colocamos 45 en la posición 4 y restamos 1 a Bucket[5] (evita pisar elementos si hubiera más con el mismo dígito).
- Para 12 (dígito 2): Posición = Bucket[2] - 1 = 1. Colocamos 12 en la posición 1 y restamos 1 a Bucket[2].
- Para 91 (dígito 1): Posición = Bucket[1] - 1 = 0. Colocamos 91 en la posición 0 y restamos 1 a Bucket[1].
- Para 23 (dígito 3): Posición = Bucket[3] - 1 = 2. Colocamos 23 en la posición 2 y restamos 1 a Bucket[3].
- Para 64 (dígito 4): Posición = Bucket[4] - 1 = 3. Colocamos 64 en la posición 3 y restamos 1 a Bucket[4].

Resultado después de ordenar por unidades: {91, 12, 23, 64, 45}

Ejemplo, paso 2: Ordenar por el siguiente dígito (decenas)

Hacemos exactamente lo mismo, pero con los dígitos de las decenas, y usando el arreglo ya ordenado por las unidades ({91, 12, 23, 64, 45}).

Resultado después de ordenar el nuevo arreglo por las decenas: {12, 23, 45, 64, 91}

Importante: en este ejemplo se trabajó con números de 2 dígitos en base 10 (fácil de entender por nuestro cerebro primate), pero esto es extrapolable a N dígitos en base b (no tan fácil de entender a la primera, a pesar de ser análogo). En este último caso, el procedimiento se deberá repetir no 2, sino N veces, y sus dígitos estarán ligados tanto a la base b considerada para su representación (número = (dígito_0 * b^0) + (dígito_1 * b^1) + ... + dígito_(N-1) * $b^{(N-1)}$), como al tamaño máximo representable en la memoria.

En el caso que ustedes deberán resolver, la base utilizada es una potencia de 2, por obvias razones.